

Yash Prakash Mishra

Full-Stack Software Engineer — TypeScript · NestJS · React · PostgreSQL

+91 84007 93384 · mishra.yash3443@gmail.com · linkedin.com/in/yash-prakash-mishra-34063016a · github.com/xcalibre1

Open to relocation to Munich — hybrid · English: professional fluency

PROFESSIONAL SUMMARY

Full-stack engineer with **5+ years delivering complex client-server systems** end-to-end: **TypeScript across the stack**, service-layer backends on **PostgreSQL**, and production React web clients. Works from a strong **object-oriented foundation** — dependency injection, module boundaries, and the Strategy, Factory, Singleton and Observer patterns — in **NestJS**, whose DI container, module system and decorators follow **Angular and Spring** closely. Owns a PostgreSQL data layer through schema design, **zero-downtime migrations** and query optimization, plus a ClickHouse platform for **data-intensive** workloads. Has shipped **APIs, SDK-style libraries** and an open-source developer tool adopted by engineering teams globally, where user feedback drives the roadmap. Led a team of 5 and set technical direction with product, design and enterprise stakeholders.

CORE SKILLS

Languages	TypeScript, JavaScript (ES6+), SQL, C++ (foundational, non-production)
Backend & Architecture	NestJS (dependency injection, modules, decorators), Express.js, Fastify, Node.js, layered service architecture, microservices, OOP & design patterns (Strategy, Factory, Singleton, Observer), HLD & LLD
Databases	PostgreSQL (data modeling, schema design, zero-downtime migrations, query optimization), MySQL, ClickHouse, MongoDB, Redis; ORM-based persistence with Sequelize and Prisma
Frontend	React.js, Next.js, Redux, TypeScript, component architecture, HTML5, CSS3, SVG
Web Performance & 3D	WebXR immersive 3D components, Webpack (route-based code splitting, tree shaking, bundle analysis), Lighthouse, time-to-interactive, graceful degradation under load
APIs & Libraries	RESTful API design, OpenAPI/Swagger, input validation & schema guards, JWT auth, webhooks, idempotent API design, reusable internal libraries, open-source developer tooling
Events & Messaging	Apache Kafka, BullMQ, AWS SQS, event-driven architecture, WebSockets (Socket.io), Server-Sent Events
Cloud & DevOps	AWS (EC2, S3, CloudFront, SQS, IAM, KMS), Docker, Kubernetes (working knowledge), GitHub Actions, Jenkins
Testing & Quality	Jest, Supertest, Mocha, unit & integration testing, Postman, code review and mentoring across a 5-engineer team
Observability	ELK Stack, structured logging, distributed tracing, on-call and incident reduction
Currently learning	Java, Spring Boot, Hibernate, PostGIS

PROFESSIONAL EXPERIENCE

Novatr — Senior Software Engineer (Backend Developer → Senior Software Engineer) **Aug 2023 – Present**
Remote · Y Combinator-backed SaaS platform, 24/7 on AWS · Led a team of 5 engineers

Stack: TypeScript, NestJS, Node.js, Express.js, React.js, Next.js, PostgreSQL, ClickHouse, Redis, Apache Kafka, BullMQ, Socket.io, Jest, Docker, AWS

Architecture & object-oriented design

- Built the platform's backend services on **NestJS** — dependency-injected providers, explicit module boundaries and decorator-based controllers — across payments, multi-tenancy, CMS and authentication. The same architectural model as Spring on the JVM.
- Architected a **provider-agnostic payments platform** integrating four external providers (Stripe, Razorpay, Affirm, Jodo) using **Strategy and Factory patterns** for runtime provider selection, so onboarding a new provider requires no change to core logic; owned each integration end-to-end from POC to production.
- Architected the B2C-to-B2B transition as a **multi-tenant** system (NestJS, PostgreSQL) with tenant-level data isolation, per-tenant configuration and enterprise onboarding workflows — **onboarded 5+ enterprise clients** as technical point of contact.
- Architected **centralised RBAC across 5+ internal tools** with role inheritance, permission scoping and audit logging (**60% fewer access incidents**), plus a multi-layer **JWT** service handling token issuance, rotation and validation across three surfaces.

PostgreSQL & data-intensive services

- Own the **PostgreSQL data layer** behind payments, tenancy and access control: schema design, data modeling, **zero-downtime migrations** and query optimization.
- Migrated years of payments history off a legacy system onto a **ClickHouse**-backed analytics platform **with no downtime**, via Node.js ETL with schema validation, integrity checks and a tested rollback path.
- Introduced Kafka **publish/subscribe pipelines** (with BullMQ for in-process jobs) for payments, user lifecycle events and transactional email, decoupling services and improving fault tolerance platform-wide.

APIs, integrations & reliability

- Designed a centralised webhook handler for all four payment providers with self-healing retry logic, event deduplication and Kafka-based status reconciliation, **cutting manual error resolution by 80%**; added circuit breakers (opossum) and backoff retries so a degraded provider fails over rather than taking checkout down with it.
- Made every payment path **idempotent**, so retries and duplicate provider callbacks cannot double-charge under concurrent load.
- Rebuilt the **data management** backend (NestJS, PostgreSQL) with content lifecycle workflows and versioning — **85% faster**, recurring on-call incidents eliminated.

Web client, real-time & technical direction

- Launched a web platform on **Next.js, React.js and TypeScript** end-to-end, and defined the **Node.js BFF API contracts** its Admin, LMS and public surfaces run on, so client needs shaped the contract rather than backend response shapes leaking into components.
- Built a real-time engine (Node.js, **Socket.io**, Redis) for live sessions, tuned for low-latency concurrent interactions with graceful degradation under load.
- Led 5 engineers across architecture decisions, **code review** and delivery timelines, **mentoring engineers at varying experience levels**; partnered with product, design and QA from discovery through production, and drove consensus with enterprise stakeholders by translating business requirements into technical architecture.

OLX Autos — Software Engineer

May 2022 – May 2023

Remote · Used-car financing infrastructure for Latin America — regulated, high-stakes workflows

Stack: Node.js, TypeScript, Express.js, PostgreSQL, MySQL, Sequelize, REST APIs, Docker, AWS

- Built and owned foreclosure and manual repayment flows in the Loan Management System, automating loan closure and repayment tracking for **10,000+ active loans in Chile** and replacing a fully manual operations process.
- Designed **loan lifecycle state machine APIs** with validation guards at every transition — permitted states only, explicit rejections, an audit record per change — **cutting manual intervention by 40%**.
- Designed risk analysis and credit scoring workflows in the Loan Origination System, **reducing credit approval time by 30%** while improving decision auditability.
- Built a **reusable PDF generation library** (PDFKit, Handlebars) consumed by the disbursement services for programmatic invoicing, saving operations 10+ hours weekly; technical point of contact for the Chile team across a 9+ hour time-zone gap.

Lenskart — Software Engineer (Frontend)

Apr 2021 – Apr 2022

India's largest eyewear retailer · Exit: project concluded on successful UAE market launch

Stack: React.js, Redux, TypeScript, Next.js, WebXR, Webpack, Node.js (BFF), REST APIs

- Integrated **WebXR immersive 3D try-on components** into the storefront, putting interactive 3D product rendering directly in the browser, alongside Google reCAPTCHA — improving engagement and reducing fraudulent checkout attempts.
- Owned and led the **complete rebuild of core commerce flows** (homepage, cart, checkout, order tracking) in **React.js, Redux and TypeScript** for the UAE market launch: **30% faster launch** with improved Lighthouse scores, as primary frontend point of contact for the client go-live.
- Applied **route-based code splitting, tree shaking and bundle analysis** (Webpack) to cut initial JS payload and improve **time-to-interactive** across every key page — the work that decides whether an asset-heavy page is usable.

SELECTED PROJECT

BullMQ Dashboard — open-source developer tooling

2025

Next.js (App Router), TypeScript, Node.js, Redis, ioredis, BullMQ, Express.js, Docker, SVG, Server-Sent Events · github.com/xcalibre1/bullmq-dashboard

- Problem:** tools such as Bull Board provide queue visibility but no operational control when a pipeline misbehaves in production — no retrigger with a corrected payload, no bulk retry, no live view.
- Built a **Next.js + TypeScript** operations dashboard with **Server-Sent Events** for live queue statistics, an editable JSON payload editor and a 24-hour throughput chart rendered in pure SVG; API routes backed by a Redis singleton (ioredis) and a cached BullMQ Queue instance for near-zero per-request overhead.
- Adopted by engineering teams globally.** Feature direction — payload field search, bulk retry, promote, pause and resume — came from **user feedback and issue reports**, which is also how the documentation and Docker Compose self-hosting setup got written.

EDUCATION

B.Tech, Computer Science — Rajkiya Engineering College, Sonbhadra

Jul 2016 – Jun 2020

ADDITIONAL

- Relocation & languages:** open to relocating to Munich for a hybrid role; professional fluency in English; 5 years across distributed teams in India, Latin America and the Middle East.
- Interests:** 3D and WebGL on the web, geospatial data, developer tooling, API design.